

ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Образовательная программа «Прикладная математика и информатика»

УДК 519.161

Отчет об исследовательском проекте на тему:
Задача о векторной упаковке контейнеров

Выполнил студент:

группы #БПМИ223, 2 курса

Наумов Леонид Андреевич

Принял руководитель проекта:

Посыпкин Михаил Анатольевич

Заведующий кафедрой, Профессор

Факультет компьютерных наук / Базовая кафедра

«Интеллектуальные технологии системного анализа

и управления» ФИЦ «Информатика и управление» РАН

Москва 2024

Содержание

Аннотация	3
1 Введение	4
1.1 Постановка задачи	4
1.2 FFD Item-centric и Bin-centric классы алгоритмов	4
1.3 Геометрические эвристики	4
1.4 Области применения	5
1.5 Основные результаты работы	5
1.6 Описание предложенного алгоритма	5
2 Описание задачи и известных алгоритмов	6
2.1 Формальная постановка задачи	6
2.2 Ход работы FFD алгоритмов	6
2.3 Выбор вектора коэффициентов	7
2.4 Ход работы геометрических эвристик	7
2.5 Вычисление веса предмета для Dot-Product	7
2.6 Вычисление веса предмета для Norm-Based-Greedy(L2)	8
3 Описание предложенного алгоритма	8
3.1 Интуиция, стоящая за предложенным алгоритмом	8
3.2 Формальное описание предложенного алгоритма	8
3.3 Алгоритмическая сложность алгоритма	9
4 Эксперименты	9
4.1 Нижняя оценка количества контейнеров	9
4.2 Описание данных и первый эксперимент	9
4.3 Вариация длин отрезков	11
4.4 Массовое тестирование	11
4.5 Сравнение с LB	12
5 Заключение	14
6 Описание литературы	14
Список литературы	15

Аннотация

В данной работе рассматривается задача векторной упаковки контейнеров (Vector Bin Packing Problem). Эта задача является обобщением задачи упаковки контейнеров, которая состоит в определении минимального числа контейнеров заданной вместимости, необходимых для размещения заданного набора предметов с известными весами, таким образом, чтобы вместимость контейнеров не была превышена. В обобщенной задаче также требуется определить минимальное число контейнеров, нужных для размещения n предметов, но, в отличие от классической постановки, каждый предмет и каждый контейнер имеют не одну, а $d > 1$ числовых характеристик (измерений). При этом, сумма по каждому из измерений соответствующих характеристик предметов, положенных в контейнер, не должна превышать его вместимость по данному измерению. Данная задача находит применение, например, в облачных вычислениях, где решается задача размещения виртуальных машин по серверам. Задача является NP-полной, то есть, нестрого говоря, нет алгоритмов её решения значительно быстрее полного перебора. По этой причине предметом изучения становятся различные эвристические алгоритмы, не дающие гарантированно оптимальный ответ, но позволяющие получать приближенные ответы за разумное время. Мною была изучена литература по данной области, реализованы некоторые из известных алгоритмов, проанализированы их слабые стороны и разработан собственный алгоритм упаковки, превосходящий известные аналоги на части тестов.

Ключевые слова

Векторная упаковка контейнеров, дискретная оптимизация, эвристические алгоритмы, жадные алгоритмы, геометрические эвристики.

1 Введение

1.1 Постановка задачи

В данной работе изучены различные эвристические алгоритмы для решения задачи векторной упаковки контейнеров (Vector Bin Packing Problem). Данная задача - это многомерное обобщение задачи упаковки контейнеров. В одномерном случае имеется конечный набор предметов с известными весами, а также контейнеры известного размера. Используя наименьшее количество контейнеров требуется разложить по ним предметы, при чем так, чтобы сумма весов предметов в каждом из контейнеров не превышала его размер. Так как существует лишь конечное количество способов разложить предметы по контейнерам (будем рассматривать только разложения, в которых в каждом контейнере лежит хотя бы один предмет), существует и такой вариант, в котором используется наименьшее количество контейнеров. В многомерном случае предметы и контейнеры представляются как d -мерные векторы. Во всех рассматриваемых далее алгоритмах каждому из предметов будет присваиваться некоторое число - вес. Алгоритмы внутри каждого из рассматриваемых ниже классов отличаются только способом присваивания веса предметам.

1.2 FFD Item-centric и Bin-centric классы алгоритмов

Предметы сортируются по убыванию весов, после чего очередной предмет помещается в первый контейнер, в котором есть достаточно для него места. У данного алгоритма нет единственного обобщения на многомерный случай, так как есть множество способов присвоения веса предмету. Таким образом одномерный алгоритм обобщается на целый класс.

В Bin-centric классе предметы также отсортированы в порядке убывания весов. В алгоритмах данного класса на очередном шаге открыт ровно один контейнер (в том числе и на первом шаге). Предмет, обладающий наибольшим весом, среди тех, которые вмещаются в открытый контейнер кладётся, помещается в него предметов. Если ни один из предметов не помещается в контейнер, то он закрывается и открывается новый. Алгоритм продолжается до тех пор, пока все предметы не будут распределены по контейнерам.

1.3 Геометрические эвристики

Геометрические эвристики являются модификацией класса Bin-centric. На каждом шаге открыт ровно один контейнер, в который также кладётся предмет с наибольшим (или наименьшим, в зависимости от реализации конкретного алгоритма) весом. Однако для опре-

деления веса предмета в расчет берется не только его размер, но и то, насколько хорошо этот предмет подходит открытому на данный момент контейнеру.

1.4 Области применения

Задача о векторной упаковке (Vector Bin Packing Problem) находит практическое применение в решении задачи о размещении виртуальных машин по серверам (Virtual Machine Placement Problem), задачи распределения ресурсов (Resource Constrained Scheduling), задачи о рюкзаке (Knapsack Problem) и других. Задача о векторной упаковке, вместе со схожей задачей о рюкзаке входит в пятёрку наиболее востребованных прикладных NP-сложных задач. [2]

1.5 Основные результаты работы

Целью проекта является разработка эффективного алгоритма оптимальной векторной упаковки контейнеров. В ходе работы на языке C++ были реализованы некоторые из известных алгоритмов, а также разработан генератор тестов. Проведено некое неформальное рассуждение, мотивирующее создание предложенного алгоритма. При помощи масштабных тестов были выявлены классы выборок, на которых данный алгоритм работает лучше остальных.

1.6 Описание предложенного алгоритма

Предложенный мною алгоритм относится к классу геометрических эвристик. На очередном шаге алгоритма для каждого предмета вычисляется каким будет вектор занятого в контейнере пространства, если положить туда этот предмет. Выборочная дисперсия координат этого вектора и является весом предмета на данном шаге. Предмет с наименьшим весом кладется в контейнер. Данный алгоритм показывает себя значительно лучше известных на небольшом классе выборок.

2 Описание задачи и известных алгоритмов

2.1 Формальная постановка задачи

В d -мерной задаче о векторной упаковке контейнеров, у нас есть набор L из n предметов $I^1, I^2, \dots, I^n$, где I^i - это d -мерный вектор с неотрицательными координатами. В распоряжении имеется достаточно большое количество контейнеров $B_1, B_2, \dots$, где B_i - это d -мерный вектор вида $(h, h, \dots, h)$, где h - фиксированное положительное число, не меньшее чем максимум по любой координате любого из предметов (иначе найдется предмет, который некуда положить). Разбиение предметов по k контейнерам называется корректным, если для любого контейнера верно, что для любой координаты $j : \sum_{i=1}^t I_j^{p_i} < h$, где $p_1, p_2, \dots, p_t$ - номера предметов положенных в этот контейнер, при этом любой предмет где-то лежит. Основная задача заключается в поиске корректного разбиения при минимизации k . Так как эта задача является NP-сложной, не существует алгоритма, значительно быстрее полного перебора, точно вычисляющего k , при любых входных данных. Так что нас будут интересовать эвристические алгоритмы, находящие приближенные к оптимальному разбиения, за полиномиальное от входных данных время.

В данной работе были выбраны две хорошо зарекомендовавшие себя геометрические эвристики Dot-Product и Norm-Based-Greedy(L2) [6], а также два FFD Item-centric алгоритма FFDSum и FFDMul.

2.2 Ход работы FFD алгоритмов

FFDSum и FFDMul - это алгоритмы, в которых предметы сначала сортируются по убыванию весов $W_1, W_2, \dots, W_n$. После, на очередном шаге кладут предмет с наибольшим весом из оставшихся в первый ящик, в котором есть достаточно места. Формулы для вычисления веса для предмета j :

$$W_{FFDSum} = \sum_{i=1}^d a_i \cdot I_i^j$$

где $a = (a_1, a_2, \dots, a_d)$ - некоторый вектор коэффициентов, выбор которого описывается в следующей главе.

$$W_{FFDMul} = \prod_{i=1}^d I_i^j$$

2.3 Выбор вектора коэффициентов

$a = (a_1, \dots, a_d)$ - некоторый вектор, определяющийся по выборке предметов. Он позволяет оценить важность вклада каждой из координат предмета. Естественной оценкой a_i является положить его равным среднему значению i координаты по всем предметам выборки $avdem_i = \frac{1}{n} \sum_{I^j \in L} I_i^j$. Мы будем рассматривать выборки, в которых для всех координат предметов значения случайно выбираются из некоего отрезка. Выбирая достаточно много элементов $avdem_i$ будет примерно равен середине этого отрезка для всех i . Таким образом вклад каждой из координат будет равным, то есть можно положить $a = (1, 1, \dots, 1)$, такой вектор и будет использоваться нами, в том числе и в FFDSum.

2.4 Ход работы геометрических эвристик

Dot-Product и Norm-Based-Greedy(L2) принадлежат к классу геометрических эвристик. Вес предметов вычисляется не в начале, а на каждом шаге алгоритма. На очередном шаге, открыт ровно один контейнер (в том числе и в начале). Каждому предмету присваивается вес, в зависимости от характеристик этого предмета и открытого контейнера. Предмет с максимальным (или минимальным, в зависимости от конкретного геометрического алгоритма) весом из тех, что помещаются в контейнер – кладется в него. Если все предметы не помещаются в контейнер, он закрывается, открывается новый и алгоритм продолжается с момента вычисления весов предметов. Алгоритм идёт до момента, пока все предметы не будут распределены по контейнерам.

2.5 Вычисление веса предмета для Dot-Product

Вес предмета в алгоритме Dot-Product вычисляется как скалярное произведение вектора оставшегося в контейнере места и вектора предмета. На каждом шаге выбирается предмет, с максимальным весом. Более строго:

$$W_{DP} = \sum_{i=1}^d I_i * (h - W_i)$$

где h - изначальный размер контейнера по всем координатам, W_i - занятое место в контейнере на координате i .

2.6 Вычисление веса предмета для Norm-Based-Greedy(L2)

Вес предмета в Norm-Based-Greedy алгоритмах вычисляется как норма разности между вектором предмета и вектором оставшегося места в контейнере. В (L2) версии алгоритма в качестве нормы используется (L2) норма, считающая покомпонентную сумму квадратов разностей между размером предмета и оставшимся местом в контейнере. Более строго:

$$W_{L2} = \sum_{i=1}^d a_i * (I_i - (h - W_i))^2$$

где h - изначальный размер контейнера по всем координатам, W_i - занятое место в контейнере по координате i , a_i определяется как в главе 2.4.

3 Описание предложенного алгоритма

3.1 Интуиция, стоящая за предложенным алгоритмом

На очередном шаге Bin-centric алгоритма заполнение контейнера означает, что для каждого из оставшихся предметов по какой-то из координат будет переполнение, если положить этот предмет в контейнер. Значит среднее значение по этим координатам довольно большое. тогда чем меньше эти координаты будут отличаться от остальных координат в коробке, тем больше места будет использовано в этой коробке. Чем больше места использовано в каждой коробке, тем меньше коробок нужно.

3.2 Формальное описание предложенного алгоритма

Данный алгоритм входит в семейство геометрических алгоритмов, ход работы которых описан в главе 2.3 Таким образом единственное, что осталось описать - это то, каким образом предметам будут присваиваться веса. Весом предмета будет считать выборочную дисперсию между координатами суммы вектора предмета и контейнера. Поэтому алгоритм далее буду называть Sample-Variance или SV алгоритмом. На очередном шаге будем класть в контейнер предмет с минимальным весом. Более строго:

$$W_{SV} = \sum_{i=1}^d (I_i + W_i - (E_{I+W}))^2$$

$$\text{где } E_{I+W} = \frac{1}{d} \sum_j (I_j + W_j)$$

3.3 Алгоритмическая сложность алгоритма

Sample-Variance, как и остальные геометрические эвристики имеет алгоритмическую сложность $O(n^2 \cdot d)$, так как всего может быть порядка n шагов, на каждом из которых нужно выбрать предмет с минимальным весом, что занимает $O(n \cdot d)$ операций. Также стоит отметить, что на каждом шаге Sample-Variance требуется $6d + 1$ операций, тогда как Dot-Product и Norm-Based-Greedy(L2) – $3d - 1$ и $4d$ операций соответственно. Эксперименты подтверждают данные оценки, SV алгоритм работает примерно в 2 раза медленнее DP и в 1.5 раза медленнее ($L2$). Для FFD алгоритмов существуют решения, с алгоритмической сложностью $O(n \cdot \log n^{d-2})$. (см. [3]). Однако неясно, работают они эффективнее наивной реализации $O(n^2)$ на разумных выборках. Например, при $n = 1000$, $d = 6$, $\log n^{d-2}$ примерно равно 10^4 , так что наивная $O(n^2)$ реализация будет эффективнее.

4 Эксперименты

4.1 Нижняя оценка количества контейнеров

Перед тем, как перейти к экспериментальным сравнениям алгоритмов, хотелось бы получить некую оценку снизу на количество используемых контейнеров. То есть найти такое число, что затратить контейнеров, меньше него точно не выйдет. Пусть $M_i = I_1^i + I_2^i + \dots + I_d^i$, где $i \leq d$. Выберем среди них максимум – M_m . Предположим, что было затрачено k контейнеров. Тогда выполнено неравенство $M_m < h \cdot k$, где h – это размер контейнера по каждой из координат. Иначе, по этой координате произошло бы переполнение, даже если считать, что в каждом контейнере расходовалось всё пространство по m координате. Пусть тогда *LowerBound* – наименьшее значение k , при котором неравенство верно. Его и будем считать нашей нижней оценкой.

4.2 Описание данных и первый эксперимент

В данной работе размер контейнера по каждой из координат всегда будет равен 1000. Количество предметов также всегда будет равняться 1000. Значения координат у векторов предметов будут выбираться абсолютно случайно с некоторого отрезка $[a, b]$. Количество измерений d будет варьироваться от 4 до 15. Каждый тест будет независимо запускаться 10 раз, результатом работы алгоритма будет считаться среднее значение результатов десяти тестов. Отрезки для генерации случайных чисел будут далее иметь вид $[h - t \cdot h, h + t \cdot h]$,

где переменная h будет меняться от 1 до 390 (на графиках значение переменной h будем откладывать на оси абсцисс), а t - коэффициент, от которого будет зависеть длина отрезков, на которых будут генерироваться значения размеров предметов. Рассматривать $h > 390$ нам будет не интересно, так как при значениях h относительно близких к 500, *LowerBound* выдаёт ответы приблизительно равные количеству предметов, так как в один контейнер очень редко будет помещаться более одного предмета. Еще до начала масштабных экспериментов

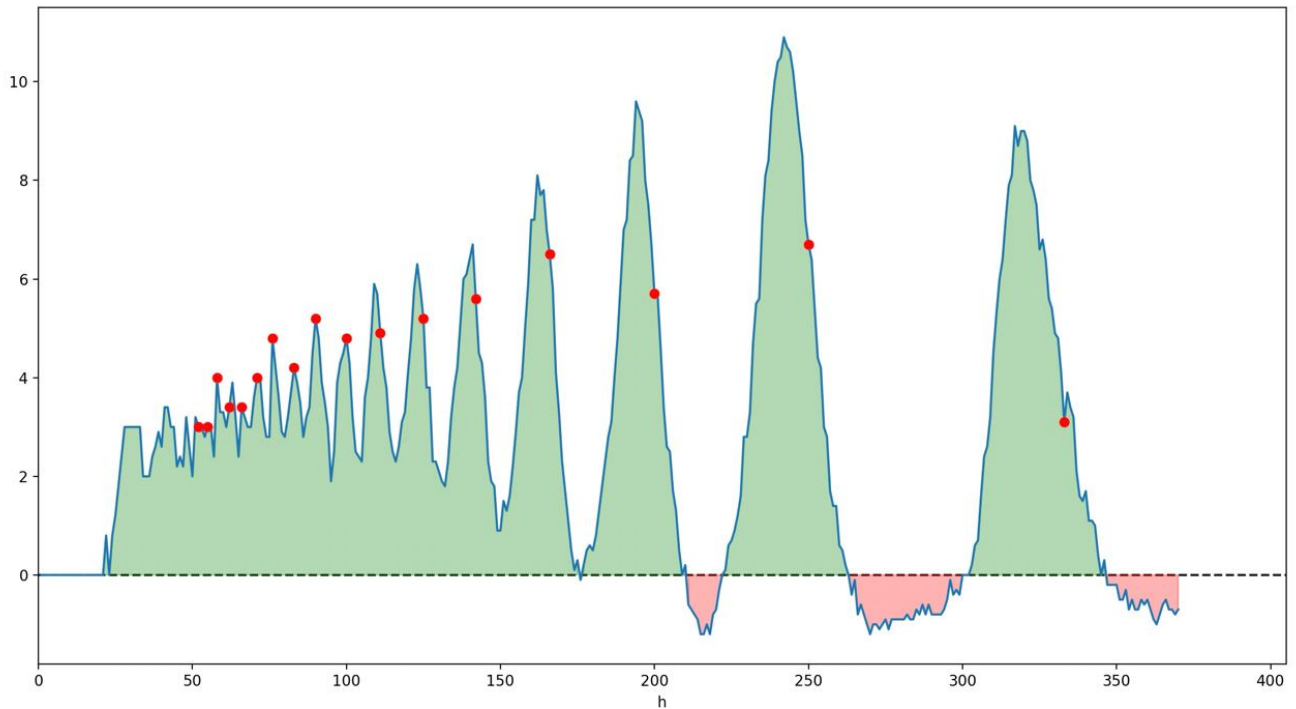


Рис. 4.1

было замечено, что *Simple – Variance* алгоритм работает лучше аналогов в окрестностях чисел вида $\frac{1000}{k}$, где k - натуральное число. Назовём такие точки вида $\frac{H}{k}$ интересными, где H - размер контейнера. Рассмотрим случай, при котором $d = 6$, а отрезок равен $[h - \frac{1}{5} \cdot h, h + \frac{1}{5} \cdot h]$. График с Рис. 4.1 показывает насколько процентов *SV* алгоритм выдаёт ответ лучше/хуже, чем лучший из ответов, выдаваемых алгоритмами *FFDSum*, *FFDMul*, *DP* и *L2* (лучшим, естественно, считается ответ, при котором число использованных контейнеров является наименьшим). Красным отмечены результаты работы алгоритма в интересных точках. Функция действительно достигает экстремумов в окрестностях интересных точек. Связано это с тем, что при $h = \frac{1000}{k}$ алгоритму удаётся разложить в каждый контейнер примерно k предметов, таким образом практически заполнив его. Когда же значение h становится больше очередной интересной точки $\frac{1000}{k}$ туда перестаёт быть возможным положить k предметов, то есть при небольшом увеличении h мы теряем целый предмет из контейнера. Также можно заметить, что при больших значениях k экстремумы несколько смещены левее относительно интерес-

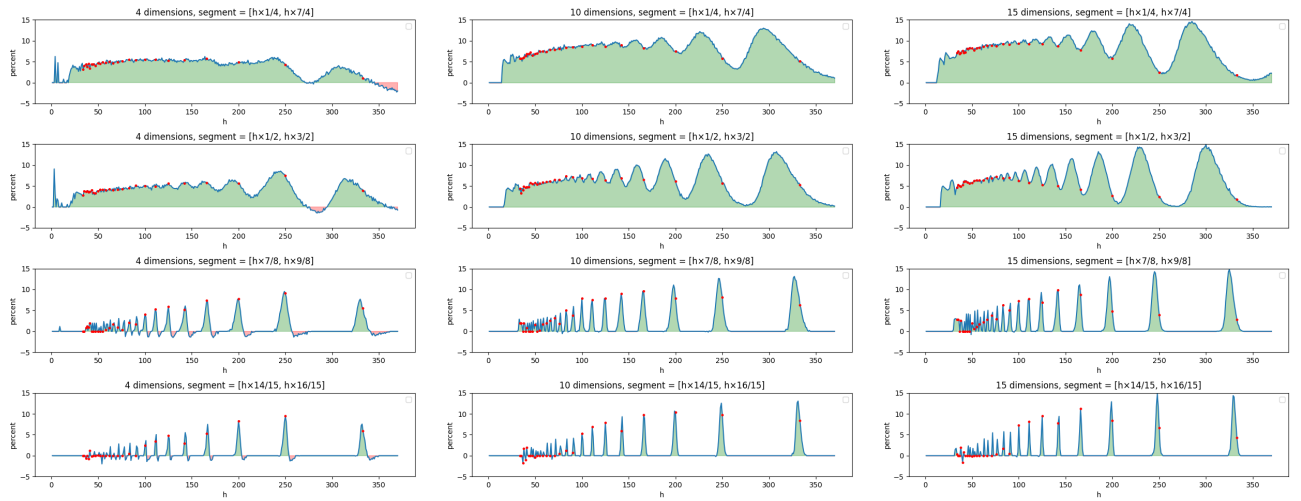


Рис. 4.2

ных точек. Это объясняется следующим рассуждением. Если брать k предметов размера h по всем координатам, то в контейнер можно положить ровно $k \cdot h$ предметов. При увеличении же разброса между размерами координат предметов ясно, что теперь k предметов в могут не влезать в контейнер угатовленного размера. Для проверки этих предположений рассмотрим примеры графиков с другими длинами отрезков выборки.

4.3 Вариация длин отрезков

По данным графикам можно заметить, что чем короче отрезок, тем сильнее результаты SV алгоритма в окрестностях интересных точек обходят другие алгоритмы, сравнительно с другими значениями h . В небольшой окрестности перед интересной точкой алгоритм работает лучше других, сразу после точки идет резкое снижение, вплоть до того, что алгоритм работает хуже одного из аналогов. Как можно заметить, размер области в которой у SV преимущество зависит от размера отрезка.

4.4 Массовое тестирование

На Рис 4.2 можно увидеть как работа алгоритма будет меняться при увеличении размерности, для наглядности предыдущие графики дублированы рядом с графиками большей размерности с соблюдением масштаба координат. Заметим, что с увеличением размерности увеличивается и процентное преимущество SV над другими алгоритмами.

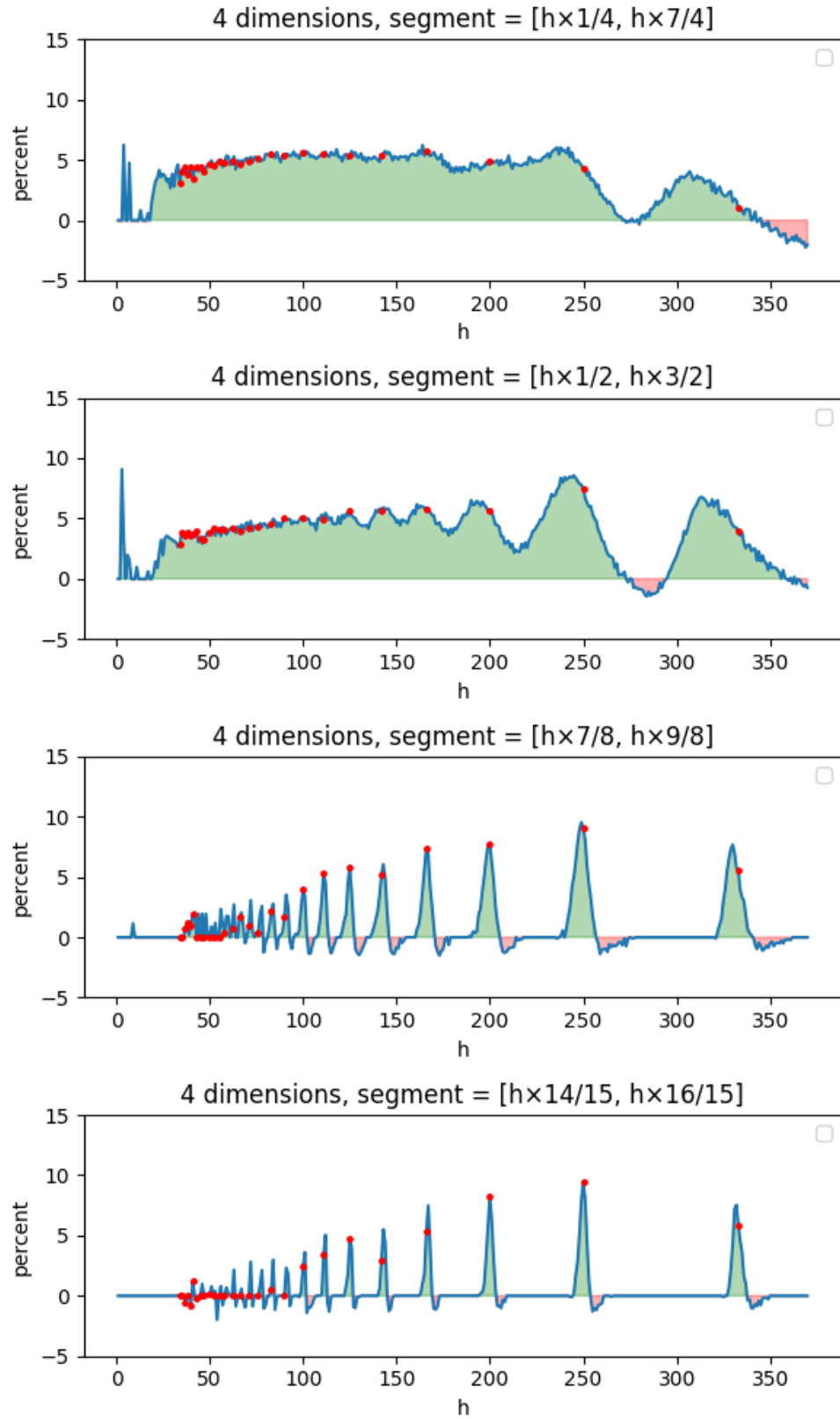


Рис. 4.3: Enter Caption

4.5 Сравнение с LB

В качестве последнего эксперимента посмотрим на графики Рис 4.4 , в которых за 100% считается оценка *LowerBound*. То есть значение точки для алгоритма *A* вычисляется

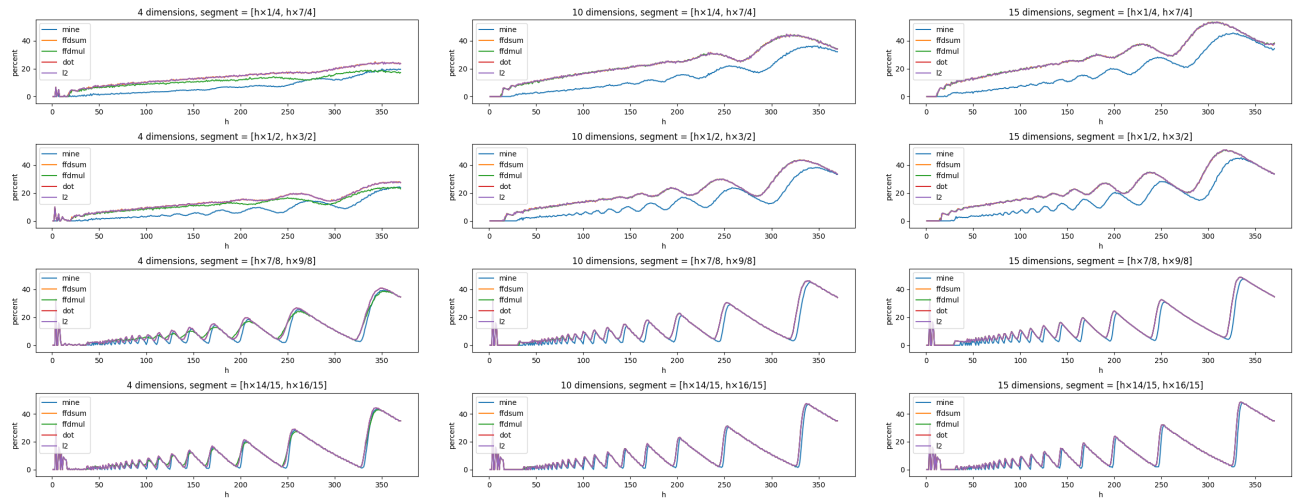


Рис. 4.4

как отношение среднего по 10 тестам количества контейнеров необходимых в A к среднему по 10 тестам значению *LowerBound*. Данные графики, в отличие от предыдущих практически не отличаются при увеличении размерности векторов. Это, предположительно, обусловлено тем, что рост ответа как у рассматриваемых эвристик, так и у *LowerBound* одинаково зависит от изменений d . Таким образом, процентная разница между двумя алгоритмами, как разница двух одинаково растущих функций будет расти также. Еще данный график показывает, что при малых размерах отрезков, все алгоритмы в окрестности точек вида $1000 - k$ дают ответы очень близкие к *LowerBound*.

5 Заключение

Заключение. Алгоритм при малых размерностях и небольших длинах отрезков выборки значительно опережает аналоги на крайне малом количестве значений h , еще реже он работает хуже. Таким образом алгоритм *Sample – Variance*, исходя из масштабных экспериментов, является отличной альтернативой ведущим известным алгоритмам на классе выборок, где вес предмета равномерно выбирается из некоторого отрезка. Особенно хорошо данный алгоритм показывает себя при решении задачи для больших значений размерности d . Однако как показывают тесты алгоритм превосходно работает на выборках, на которых размеры поступают равномерно с достаточно больших отрезков.

6 Описание литературы

[6] описаны геометрические эвристики, являющиеся отличной альтернативой для вариаций FFD. Здесь можно посмотреть на каких наборах данных стоит использовать вариации FFD, а на каких лучше работают геометрические эвристики. В этой статье также были впервые предложены алгоритмы Dot-Product и Norm-Based-Greedy(L2), с которыми в дальнейшем будет сравниваться мой алгоритм.

В [4] содержится информация о прикладной области в которой применяется векторная упаковка, наряду с описанием известных алгоритмов, проблемы и специфика применения методов векторной упаковки в прикладных задачах.

В [5, Глава 8] описывается одномерный случай задачи, эвристики на нём, методы точного вычисления ответа, вычислительные эксперименты, оценки качества работы алгоритмов и описание алгоритмов редукции, про сравнение работы этого алгоритма можно почитать здесь [1, с. 61—75]. В этой книге также даны замечания по актуальности использования редукции для алгоритмов и наборов данных.

В [2] описывается обобщение задачи, в котором рассматриваются контейнеры с переменными размерами, это обобщение находит своё применение для решения Machine Reassignment Problem, а также моделирования Virtual Machine Placement Problem.

Список литературы

- [1] В: *Systems and Means of Informatics* (нояб. 2023). ISSN: 0869-6527. DOI: [10.14357/08696527230305](https://doi.org/10.14357/08696527230305). URL: <http://dx.doi.org/10.14357/08696527230305>.
- [2] Michaël Gabay и Sofia Zaourar. “Vector bin packing with heterogeneous bins: application to the machine reassignment problem”. В: *Annals of Operations Research* 242.1 (сент. 2015), с. 161—194. ISSN: 1572-9338. DOI: [10.1007/s10479-015-1973-7](https://doi.org/10.1007/s10479-015-1973-7). URL: <http://dx.doi.org/10.1007/s10479-015-1973-7>.
- [3] *Geometric range searching / ACM Computing Surveys* — doi.org. <https://doi.org/10.1145/197405.197408>. [Accessed 25-04-2024].
- [4] A. Ignatov, I. Maslova, M. Posypkin, W. Yang и J. Wu. “A two-phase heuristic algorithm for power-aware offline scheduling in IaaS clouds”. В: *Journal of Parallel and Distributed Computing* 178 (апр. 2023), с. 1—10. ISSN: 0743-7315. DOI: [10.1016/j.jpdc.2023.03.006](https://doi.org/10.1016/j.jpdc.2023.03.006). URL: <http://dx.doi.org/10.1016/j.jpdc.2023.03.006>.
- [5] J.B.J.M. De Kort. “Knapsack problems, algorithms and computer implementations: Silvano Martello and Paolo Toth Wiley, Chichester, England, 1990”. В: 1991. URL: <https://api.semanticscholar.org/CorpusID:118235143>.
- [6] Rina Panigrahy, Kunal Talwar, Lincoln Uyeda и Udi Wieder. “Heuristics for Vector Bin Packing”. ЯНВ. 2011. URL: <https://www.microsoft.com/en-us/research/publication/heuristics-for-vector-bin-packing/>.
- [7] Wikipedia contributors. *Bin packing problem* — *Wikipedia, The Free Encyclopedia*. [Online; accessed 7-February-2024]. 2024. URL: https://en.wikipedia.org/w/index.php?title=Bin_packing_problem&oldid=1193855231.